Lecture 19: TCP Part 1

Connections, Sequencing, and Flow Control

EC 441 — Introduction to Computer Networking
Boston University

Spring 2026

Learning Objectives

By the end of this lecture, you should be able to:

- Identify and explain the purpose of each field in a **TCP segment header**
- Explain the **byte-stream model** and why sequence numbers count bytes, not segments
- Trace the **3-way handshake** and explain ISN randomization
- Describe **connection teardown** (FIN/ACK, RST, TIME_WAIT)
- Explain TCP's **RTT estimator** as a discrete-time low-pass filter, and why variance tracking (RTTVAR) is necessary for a robust RTO
- Explain **fast retransmit**: why three duplicate ACKs signal loss
- Describe **flow control**: how rwnd prevents receiver buffer overflow

Key Acronyms

Acronym	Meaning
ACK	Acknowledgment
EWMA	Exponentially Weighted Moving Avg
FIN	Finish (close connection)
ISN	Initial Sequence Number
MSL	Maximum Segment Lifetime
MSS	Maximum Segment Size

Acronym	Meaning
RTO	Retransmission Timeout
RTT	Round-Trip Time
RST	Reset
rwnd	Receive Window
SACK	Selective ACK
SRTT	Smoothed RTT (filtered estimate)

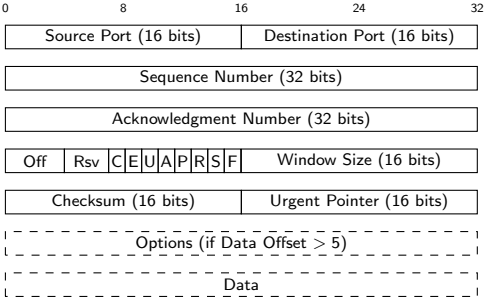
From L18 Building Blocks to TCP

In L18 we built the RDT toolkit from scratch. TCP is the deployed answer.

L18 concept	**TCP's answer**
Sequence numbers	Byte offsets in the stream (not packet counts)
Cumulative ACK (GBN)	Default; SACK option adds SR-style individual ACK
Receiver buffer (SR)	Required; advertised as the receive window (rwnd)
Pipelining	Effective window = min(cwnd, rwnd)
GBN single timer	One retransmit timer + fast retransmit (3 dup ACKs)
Full-duplex	Two independent byte streams, one per direction

Today: segment structure, connections, reliable delivery, flow control.
L20: congestion control (cwnd).

TCP Segment Header (20 bytes minimum)



Flags: **C**WR, **E**CE (ECN), **U**RG, **A**CK, **P**SH, **R**ST, **S**YN, **F**IN

Header Fields: Sequence, ACK, Flags, and Offset

Sequence Number (32 bits)
Byte offset of the *first byte of data* in this segment.
SYN and FIN each consume one sequence number.

Acknowledgment Number (32 bits)
Next byte the sender of this segment *expects to receive* (cumulative ACK).
ACK $= N$ means "I have all bytes through $N - 1$."
Valid only when ACK flag $= 1$.

Data Offset (4 bits)
Header length in 32-bit words (like IHL in IPv4).
Min $= 5$ (20 bytes); max $= 15$ (60 bytes).

Key flags

Flag	Meaning
SYN	Connection initiation; carries ISN
ACK	ACK number field is valid
FIN	Sender has no more data
RST	Abort connection immediately
PSH	Deliver to app without buffering
URG	Urgent pointer is valid
CWR/ECE	Congestion signals (L20)

Header Fields: Window, Checksum, and Options

Window Size (16 bits) — rwnd
Free buffer space at the receiver.
Sender must keep bytes-in-flight $\leq$ rwnd.
Limits to 64 KB without Window Scale option.

Checksum (16 bits)
Covers pseudo-header (src IP, dst IP, proto=6, TCP length) + header + data.
Mandatory in both IPv4 and IPv6 (unlike UDP's optional checksum).

Key TCP options

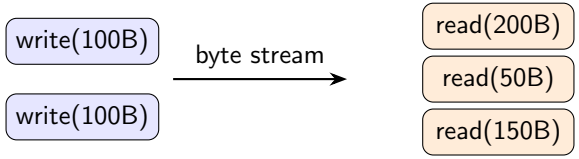
Option	Purpose
MSS	Max segment size; exchanged at SYN
Window Scale	Multiplies window by 2^n; up to 1 GB
SACK	SR-style: ACK specific byte ranges*
Timestamps	Precise RTT; removes Karn ambiguity

MSS on Ethernet: 1460 bytes
(1500 MTU − 20 IP − 20 TCP)

*SACK is on by default in Linux, macOS, Windows. In Wireshark you will almost always see SACK-based streams. To observe classic cumulative-ACK behavior, disable with `sysctl -w net.ipv4.tcp_sack=0` (Linux) or `net.inet.tcp.sack=0` (macOS).

The Byte-Stream Model

TCP delivers a **continuous byte stream** — there are no message boundaries at the TCP level.



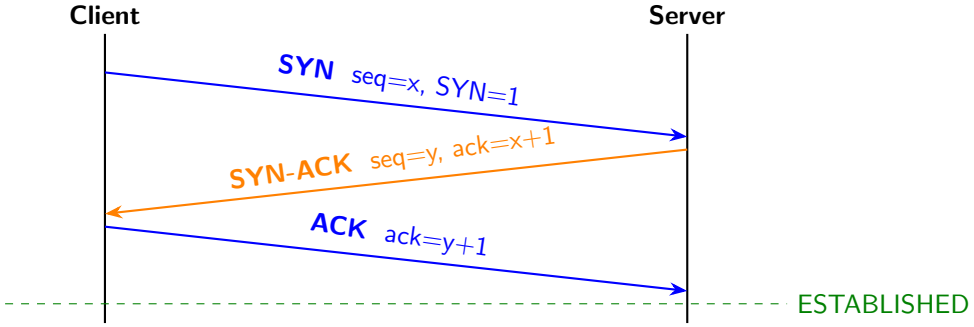
Sequence numbers count bytes, not segments.

- ISN = 1000; first segment carries 500 bytes of data
- Seq = 1000; payload = bytes 1000–1499
- Next segment: seq = 1500
- ACK from receiver: ack = 1500 ("send me byte 1500 next")

Applications that need message framing must impose it themselves (HTTP uses blank lines; TLS records carry explicit lengths).

3-Way Handshake

Both sides must agree on ISNs and negotiate options *before* data flows.
Minimum messages for reliable mutual agreement: **three**.



SYN and SYN-ACK each consume one sequence number (even with no data payload)
so that loss of either can be detected and the segment retransmitted.

ISN Randomization

Why not start every connection at seq = 0?

Problem 1: Stale segments

A connection closes. A new connection reuses the same 5-tuple and starts at seq=0. A delayed segment from the *old* connection arrives with a low sequence number — it falls in the new connection's window and is accepted as valid data.

Randomizing the ISN makes the probability of overlap negligible.

Problem 2: Security

Predictable ISNs allow an attacker to forge TCP segments:

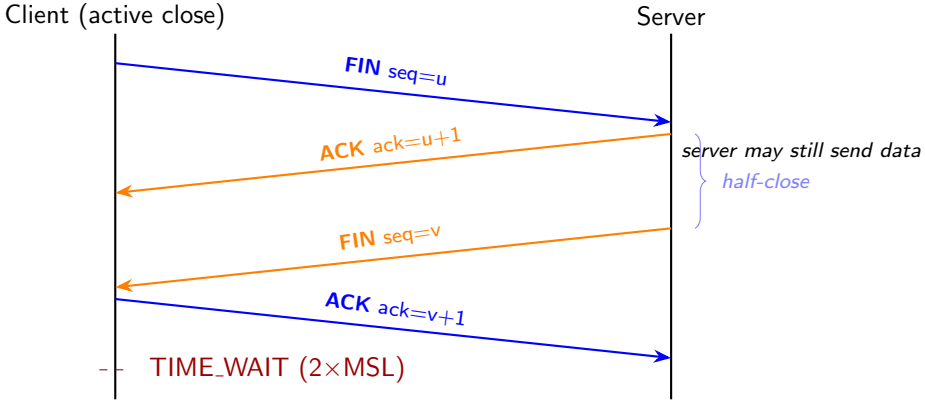
- Attacker cannot see the SYN-ACK (off-path)
- But if ISN is predictable, attacker can guess the correct ACK value
- Allows injection of data into a connection the attacker can't observe

Randomization defeats sequence-prediction attacks.

Modern Linux: ISN = MD5 hash of (src IP, dst IP, ports, key, timestamp) — cryptographically unpredictable.

Connection Teardown: FIN/ACK Exchange

Each direction closes independently ("half-close").



FIN consumes one sequence number. Server ACK and FIN are often combined if no more data to send.

TIME_WAIT and RST

TIME_WAIT (active closer only)
Duration: $2 \times$ MSL (typically 2–4 minutes)
Why it must exist:

1. Final ACK may be lost
   Server retransmits FIN.
   Client must still be alive to re-ACK.
2. Prevent 5-tuple reuse confusion
   Stale segments from the old connection must expire before a new connection reuses the same addresses and ports.

`ss -t -a` on a busy server shows hundreds of TIME_WAIT sockets — this is normal.

RST: Abrupt close
Sent when:

- Connection request arrives at a port with no listener
  ("Connection refused")
- One side crashed and restarted — half-open detection
- Firewall terminating a connection

RST is **not acknowledged**.
Receiver immediately discards all buffered data and closes the socket.
No TIME_WAIT.

Demo: Observing TCP Connections

```
# All TCP connections and their states
ss -t -a -p

# Open a raw TCP connection to BU's web server
nc -v bu.edu 80

# In a second terminal while nc is open:
ss -t -p # see ESTABLISHED state

# TYPE (then press Enter twice):
# GET / HTTP/1.0
#
# Watch the response arrive, then observe FIN teardown
# After close, re-run ss to see TIME_WAIT
```

Things to observe:

- The ESTABLISHED 5-tuple (local addr:port $\leftrightarrow$ remote addr:port)
- State transitions as the connection closes
- TIME_WAIT socket lingering after nc exits

Reliable Delivery in TCP

Cumulative ACKs (GBN-style)

ACK $= N \Rightarrow$ "I have all bytes through $N-1$."

- One ACK can acknowledge many bytes
- **Delayed ACK**: receiver waits up to 500 ms (or 2 segments) before ACKing — reduces ACK traffic
- SACK option adds SR-style selective acknowledgment for specific byte ranges

Retransmission timer (RTO)

One timer per connection, tracking oldest unACKed byte.

- RTO fires $\Rightarrow$ retransmit; double the RTO (backoff)
- RTO must track changing network RTT
- **Too short**: spurious retransmits, wastes bandwidth
- **Too long**: slow recovery from real loss

The right RTO depends on RTT — which must be *estimated*.

RTT Estimation: EWMA as a Low-Pass Filter

TCP tracks RTT with an **Exponentially Weighted Moving Average**:

$$\text{SRTT} \leftarrow (1-\alpha)\cdot \text{SRTT} + \alpha \cdot R \qquad \alpha = \tfrac{1}{8}$$

Engineers: this is a first-order IIR low-pass filter.
Expanding the recursion:

$$\text{SRTT}_n = \alpha \sum_{k=0}^{n} (1-\alpha)^k \cdot R_{n-k}$$

Past samples are weighted by a geometrically decaying envelope with ratio $(1-\alpha)$.

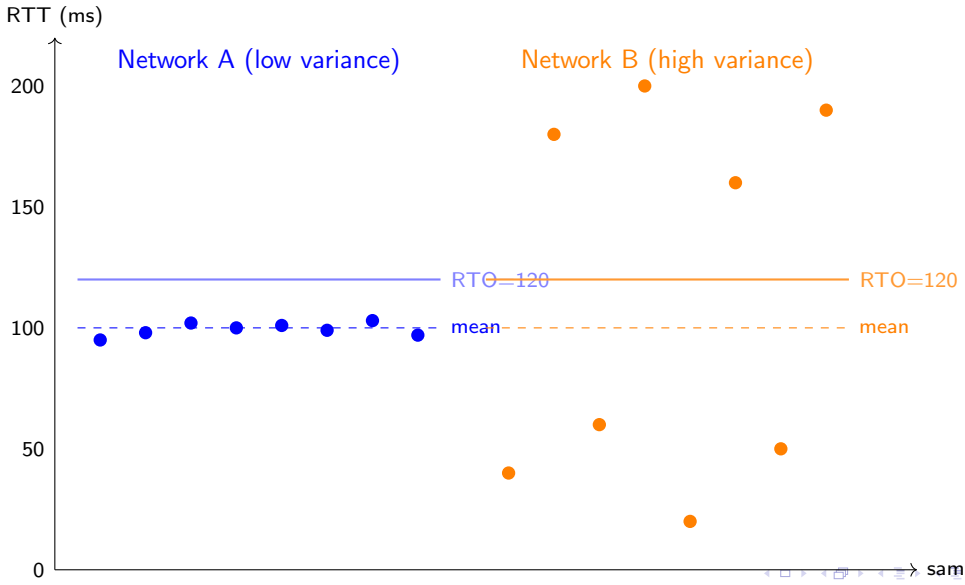
Time constant $\approx 1/\alpha = 8$ samples
(weight falls to $1/e$ in 8 RTT measurements)

Why $\alpha = 1/8 = 2^{-3}$**?**
Multiply becomes a right-shift.
Efficient on 1980s hardware.
Still a practical tradeoff today.

Large $\alpha \Rightarrow$ fast response, noisy estimate — Small $\alpha \Rightarrow$ smooth, sluggish

Why the Mean Is Not Enough: Variance Matters

Two networks, both with mean RTT = 100 ms:



Jacobson's Fix: RTTVAR and the RTO Formula

Van Jacobson (1988): track the **mean absolute deviation** of the RTT:

$$\text{RTTVAR} \leftarrow (1-\beta)\cdot\text{RTTVAR} + \beta\cdot|\text{SRTT} - R| \qquad \beta = \tfrac{1}{4}$$

RTTVAR is another EWMA — this one tracking **variability** (noise power).

$$\boxed{\text{RTO} = \text{SRTT} + 4\cdot\text{RTTVAR}}$$

Engineering interpretation:
RTO = mean + 4×variability
A confidence-interval bound that *adapts* to network conditions. For a Gaussian signal, $\pm 4\sigma$ gives 99.994% coverage. RTT distributions are right-skewed (can't go negative), so 4×MAD is a practical safety margin.

The two-filter system:

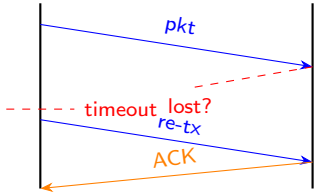
SRTT	tracks the signal mean
RTTVAR	tracks the noise power
RTO	mean + 4×noise

$\beta = 1/4 = 2^{-2}$: another shift.
This formula is in Linux TCP today.

Karn's Algorithm and Exponential Backoff

Karn's algorithm

When a retransmit occurs, an ACK arrives — but which transmission did it acknowledge?



Which tx? **Unknown.**

Rule: Do not update SRTT/RTTVAR from any ACK following a retransmit.

Timestamps option removes this ambiguity entirely.

Exponential backoff

Each consecutive timeout:

$$\text{RTO} \leftarrow 2 \times \text{RTO}$$

(up to a maximum, typically 60–120 s)

Why? If the network is dropping packets because it is **overloaded**, sending faster makes it worse.

Same principle as Ethernet CSMA/CD (L07) — back off to reduce contention.

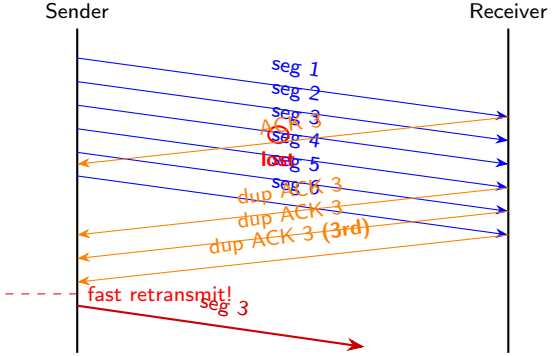
After a successful transmission, RTO resets to the computed value.

Fast Retransmit: Don't Wait for the Timer

The RTO can be hundreds of ms. Can we detect loss *sooner*?

When a receiver gets an **out-of-order** segment, it immediately sends an ACK for the last in-order byte — a **duplicate ACK**.

Rule: **3 duplicate ACKs** ⇒ retransmit immediately, without waiting for RTO.

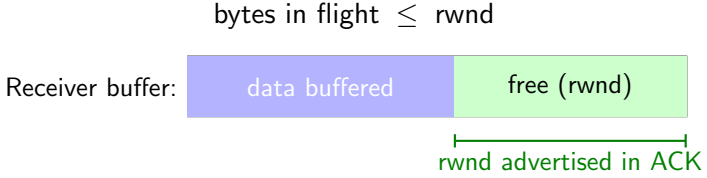


Why 3 (not 1)? One or two dup ACKs can arise from normal IP reordering. Three is a reliable loss signal.

Flow Control: The Receive Window

Problem: sender and receiver run at different speeds. If the sender transmits faster than the application reads, the receive buffer overflows.

Solution: every TCP segment carries **rwnd** — the receiver's available buffer space.



As the **application reads** data from the buffer, rwnd increases and is advertised in the next ACK — opening the window and allowing more data in.
If rwnd = 0: sender **stops**, then periodically sends a **1-byte zero-window probe** to check whether space has opened.

Flow Control: Worked Example

Receiver has a 16 KB buffer. Application is reading slowly.

After 3 segments arrive (1460 B each):
Data buffered: $3 \times 1460 = 4380$ B
Free space: $16384 - 4380 = 12004$ B
$\Rightarrow$ ACK carries **rwnd = 12004**

Application reads 4380 B:
Data buffered: 0 B
Free space: 16384 B
$\Rightarrow$ ACK carries **rwnd = 16384**

Buffer fills completely:
$\Rightarrow$ ACK carries **rwnd = 0**
Sender pauses; sends 1-byte probes until receiver reopens window

Nagle's Algorithm (brief)

Problem: interactive applications (SSH, Telnet) send many tiny writes. Each keystroke becomes a 41-byte segment (1 byte data + 40 bytes headers) — wasteful.

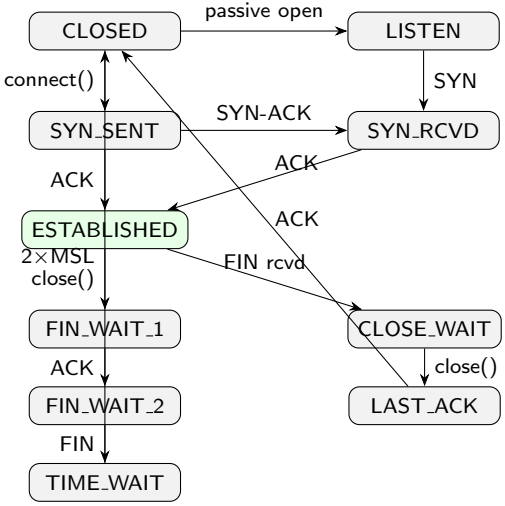
Nagle's algorithm: if there is data already in flight (unACKed), **buffer small writes** until an ACK arrives, then send everything accumulated as one segment.

Benefit:	**Cost**:
Reduces small-packet flooding on slow or congested links.	Adds latency — the buffered data waits for an ACK.

Applications that *cannot* tolerate this delay disable it with `TCP_NODELAY`:

- VoIP, online games, interactive SSH
- Any protocol where low latency > bandwidth efficiency

TCP State Machine (key states)



`ss -t -a` shows sockets in all states. Many TIME_WAIT entries on a busy server is normal.

Bridge to L20

What we covered today — TCP's core mechanisms:

- Segment structure and the byte-stream model
- Connection setup (3-way handshake, ISN randomization) and teardown (FIN, RST, TIME_WAIT)
- Reliable delivery: cumulative ACKs, RTO with adaptive estimation (EWMA + RTTVAR), fast retransmit
- Flow control: rwnd prevents receiver buffer overflow

What's missing: the sender also needs to avoid overloading the *network* — not just the receiver's buffer.

Effective window = **min(cwnd, rwnd)**

L20: congestion control — slow start, AIMD, fast recovery, CUBIC, QUIC.